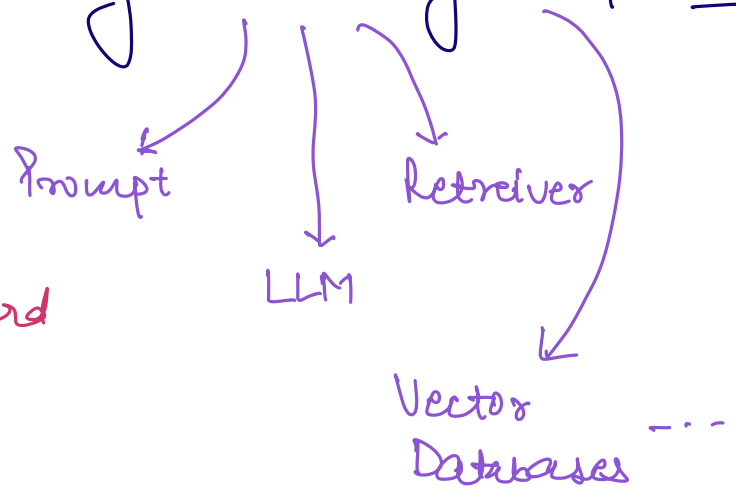


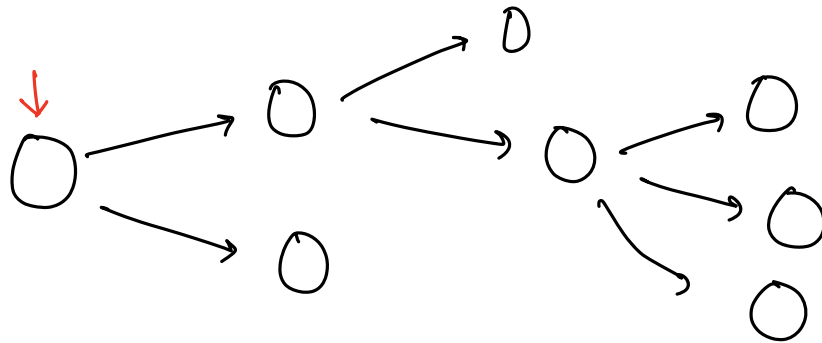
LangChain :

→ It is a framework which helps us to build LLM based applications by connecting components.

→ Mostly used for building simple & straightforward applications.



LangGraph.



→ It is a framework which helps to build complex agent workflows.

→ Instead of thinking in terms of simple chains, here we think in terms of Nodes, Edges, States, branching, loops & human in the loop feedback loop.

LangChain ⇒ Build LLM based application faster.

LangGraph ⇒ Build controlled, complex, stateful & more reliable agentic workflow.

Area	LangChain	LangGraph
Main purpose	Connect LLMs with prompts, tools, retrievers, memory, output parsers	Orchestrate complex agent workflows using graph-based control flow
Flow style	Usually linear or high-level agent loop	Explicit graph: nodes + edges + conditions
Best for	RAG apps, chatbots, simple tool-calling agents, quick prototypes	Multi-step agents, approval flows, retry loops, stateful apps, complex decision-making
Control	Less explicit control over every step	More control over what happens next
State management	Can use memory, but flow state is not always central	State is a core idea; each node reads/writes shared state
Loops / cycles	Possible, but can become harder to manage manually	Built for loops, retries, reflection, and repeated tool usage
Human-in-the-loop	Possible, but needs custom handling	First-class use case in LangGraph-style workflows
Production reliability	Good for many apps, especially simple flows	Better when you need persistence, checkpointing, recovery, and traceable execution

When should we use langchain?

→ RAG application

→ ChatBot

→ Agentic App with basic tool calling.

When should we use langgraph?

→ Multiple paths possible

→ Branching

→ Loops or Retries.

→ Multiple tools.

→ Stateful workflows.

→ Long running tasks.

#

langgraph is a low-level orchestration framework for building long-running, stateful & complex workflows or Agents.

It doesn't force one fixed agent architecture, instead it gives building blocks like nodes, states, edges etc.

State.

→ Memory of the graph persisted during execution.

```
Class SupportedState(Dict):
```

```
    user-query: str,
```

```
    category: str,
```

```
    answer: —,
```

```
    —: —
```

```
    —: —
```

```
    —: —
```

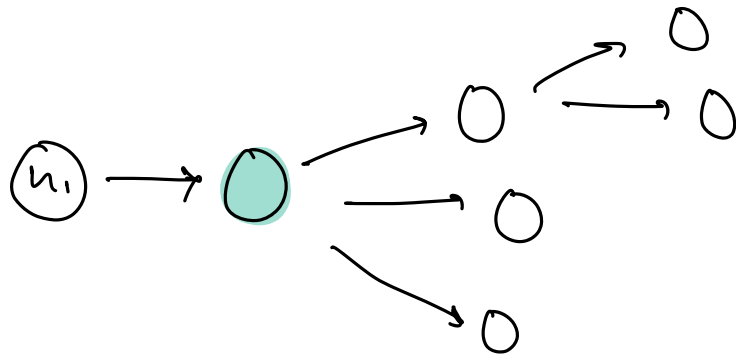
```
    —: —
```

→ In langgraph, nodes usually do not return the full state, they only return the part of the state they want to update.

→ If a node returns only one field, langgraph applies that as an update to the existing state rather than creating the complete state from scratch.

Nodes.

→ A node is just a python function.



```
def book-flight(--):
```

```
=====
```

A node can do multiple things. :

→ Call an API

→ Call LLM

→ query a DB

→ execute some business logic.

LangGraph nodes are python functions that receives some inputs, executes the functionality & returns the updates.

Nodes can be synchronous or asynchronous.

Edge.

→ Defines the flow.

Edge

- Normal Edge $\Rightarrow$ Simply goes from one node to the next node.
- Conditional Edge
 - Decides next node based on some condition.

